An Optimal Deterministic Adversary for Sorting

Nicholas Nash
ORCID: 0009-0003-0475-7852

8 November 2026

Abstract

This paper describes an optimal *deterministic*, polynomial-time adversary for ordinary sorting. For any correct sorting algorithm, the adversary forces that algorithm to make $(1 - o(1))n \log_2 n$ comparisons. The best previously known adversary forced $\frac{37}{64} n \log_2 n$ comparisons. The adversary recursively moves items into ordered subproblems, extending the classical framework in which items move down a binary tree.

Methodological disclosure

Except for the abstract, the text of this paper is AI-generated using GPT 6 Astra. The author is working on a clear, human-readable exposition.

1 Introduction

We consider comparison sorting of n distinct labelled keys. A sorter chooses each pair using the answers to its earlier comparisons and must be correct for every strict total order on the labels. A sorting adversary supplies those answers as the comparisons arrive. It is *legal* if every finite sequence of its answers is consistent with at least one total order. The adversary knows n and sees the comparisons, but need not inspect the sorter that generates them. Throughout, log denotes $\log_2$, and ln the natural logarithm; we assume $n \geq 2$.

The classical sorting lower bound guarantees the existence of a hard input for every correct sorter. Here the problem is to produce hard answers by one *deterministic, polynomial-time* procedure. The polynomial bound applies to the adversary's computation; the comparison lower bound applies to the sorter. Without the time restriction, a deterministic optimal adversary is immediate. With randomness, polynomial-time adversaries achieve the optimal asymptotic bound in expectation and with high probability.

Without the time restriction. Keep all total orders consistent with the answers so far. Each comparison partitions them into two classes; answer so as to retain a larger class. After q comparisons, at least $n!/2^q$ orders remain, forcing $q \geq \log(n!)$. This rule is deterministic, but enumerating its classes may require factorial work. A consistent total order is a *linear extension* of the partial order given by the answered comparisons; counting these extensions exactly is #P-complete [8].

Deterministic adversaries from graph entropy. Kahn and Kim [10] already give a deterministic polynomial-time adversary for sorting with partial information. Starting from any given partial order P, it answers comparisons online and forces $\Omega(\log e(P))$ comparisons, where $e(P)$ is the number of linear extensions of P. Their method uses graph entropy and convex

optimization via the ellipsoid method. For ordinary sorting, the initial partial order has no relations, so $e(P) = n!$ and their guarantee is $\Omega(n \log n)$. Thus deterministic polynomial-time adversaries are available even for arbitrary initial partial information. The remaining question for ordinary sorting is whether such an adversary can force $(1 - o(1))n \log n$ comparisons.

With randomness. An efficient adversary can simply choose a uniformly random permutation of the labels, keep it hidden, and answer from the ranks in that permutation. For every fixed correct sorter, the usual comparison-tree argument gives an expected cost of at least $\log(n!)$. It also gives a high-probability bound. At most 2^q permutations can terminate at leaves of depth at most an integer q, so the comparison count C satisfies

$$\mathbb{P}(C \le q) \le \frac{2^q}{n!}.$$

Thus $C \ge \log(n!) - O(\log n)$ with probability $1 - o(1)$. Generating an exactly uniform permutation takes expected polynomial bit time; subsequent answers require only rank lookups. A fixed polynomial time bound is obtained by using an almost uniform permutation, with a vanishing additional failure probability. These guarantees also hold for an always-correct randomized sorter whose random bits are independent of the adversary's: condition on the sorter's random choices and apply the same argument.

For any prescribed permutation, a correct sorter can first check its $n - 1$ consecutive pairs, finish if all checks agree, and otherwise use a general sorting algorithm. That permutation then costs only $n - 1$ comparisons. The deterministic adversary must choose its answers in response to the questions it receives.

Randomized approximation of the number of linear extensions. Randomization also makes it possible to estimate how many linear extensions remain after each possible answer. Given the current partial order P, assign each label a real coordinate in $(0, 1)$ and form the region

$$K(P) = \{X \in (0,1)^n : X_i < X_j \text{ whenever } i <_P j\}.$$

Each allowed total order occupies volume $1/n!$ in the unit cube. Hence $\mathrm{vol}(K(P)) = e(P)/n!$ [7]. The two possible answers to a comparison cut this region into two parts, and their volumes are proportional to their counts of allowed orders.

Dyer, Frieze, and Kannan [9] give a randomized polynomial-time algorithm for relative volume approximation and explicitly note its application to linear extensions. To use it here, first check whether the queried relation is already forced by P. If so, return it directly. Otherwise both directions are feasible and give positive-volume regions. Estimate their volumes and choose the larger estimate. When both estimates have relative error at most ε, the chosen part retains at least $(1 - \varepsilon)/2$ of the current volume. The counting argument then forces

$$q \ge \frac{\log(n!)}{1 - \log(1 - \varepsilon)} = (1 - O(\varepsilon)) \log(n!)$$

comparisons. Taking $\varepsilon = 1/\lceil \log(n + 1) \rceil$ gives $n \log n - O(n)$.

For a high-probability guarantee, allow $Q = 2n \lceil \log(n + 1) \rceil$ queries before fixing a consistent order. Amplify each volume estimate to failure probability at most n^{-4}. Each call uses fresh randomness, so this bound holds conditional on the preceding history. A union bound over at most $2Q$ estimates gives total failure probability $O(\log n / n^3)$, even for adaptive queries. These accuracy requirements preserve polynomial time. Every run remains legal, including when an estimate fails, because only feasible answers are offered.

The issue is therefore *derandomization*: obtain the optimal asymptotic comparison bound with a single deterministic response rule whose work per comparison is polynomial in n. Kaplan, Zamir,

and Zwick [5, Section 7] explicitly ask for an adversary forcing $(1 - o(1))n \log n$ comparisons. Our main theorem answers this question.

Theorem 1.1. *There is a deterministic legal adversary, initialized using only n, that performs a fixed polynomial number of bit operations per comparison and forces every correct comparison sorter to make*

$$(1 - o(1))n \log n$$

comparisons.

Merge sort uses at most $n \lceil \log n \rceil$ comparisons, so this bound is asymptotically optimal. The derandomization uses a restricted recursive representation of the remaining orders. The structure of these regions allows the adversary to calculate the required volume estimates deterministically in polynomial time. Its running-time guarantee is asymptotic and involves very large parameter thresholds.

Section 2 reviews the classical tree adversaries and the construction of Kaplan, Zamir, and Zwick. Section 3 describes our state and comparison procedure. Section 4 proves the comparison bound using two lemmas on probability estimation and anchor selection, whose proofs are deferred. Section 5 establishes the refinement properties and the geometric bound at termination; Section 6 supplies the estimator. Section 7 proves polynomial running time. The geometric density estimates and the accuracy proof for finite averaging are in the appendices. The manuscript source is available online.[1]

2 Related Work

2.1 Binary-tree adversaries

Atallah and Kosaraju [1] gave an efficient sorting adversary with a lower bound of $\frac{1}{2}n \log n$ comparisons. Related adversaries occur in the work of Borodin, Guibas, Lynch, and Yao [2] on searching after partial sorting, Richards and Vaidya [3] on the distribution of comparisons among keys, and Brodal, Chaudhuri, and Radhakrishnan [4] on maintaining the minimum. The binary-tree rule of Borodin et al. and Brodal et al. provides the starting point for our construction.

Place every item at the root of an ordered binary tree. Items in disjoint subtrees have the left-to-right order of those subtrees. A comparison changes the placement as follows.

- If both items occupy the same node, move one to its left child and the other to its right child, and answer accordingly.
- If one item is at an ancestor of the other, move the higher item to the child whose subtree does not contain the lower item.
- If their nodes are unrelated by ancestry, answer from their left-to-right order, without moving either item.

For a concrete example, start with A, B, C, D at the root. Answering $A < B$ moves A to the left child and B to the right, while C, D remain at the root. If the sorter next compares A and C, the ancestor rule moves C to the right child and gives $A < C$. Comparing D and B then moves D to the left child and gives $D < B$. The two children now contain $\{A, D\}$ and $\{B, C\}$. A subsequent comparison of D with C is answered $D < C$ without moving either item. The orders within $\{A, D\}$ and within $\{B, C\}$ remain unresolved. Figure 1 shows the placements.

Each comparison increases the sum of item depths by at most two. At completion, occupied nodes must be distinct and unrelated by ancestry: otherwise two represented items can occur in either relative order, leaving an unresolved comparison. Associate the root with $(0, 1)$, and

[1] https://github.com/nicknash/sorting-adversary-proofs/tree/main/for-humans

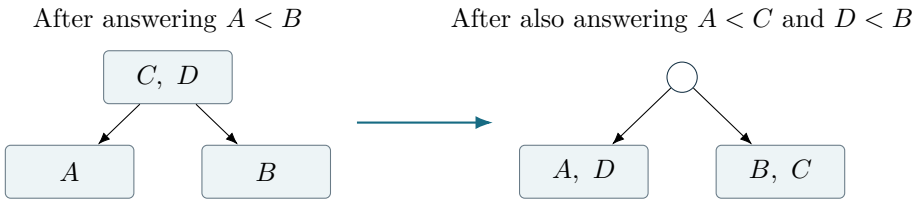


Figure 1: The binary-tree adversary. The first comparison separates two items at the same node. The next two comparisons each move an item from an ancestor into the opposite child. In the final state, comparing D with C requires no movement.

associate each child with the corresponding half of its parent's interval. A node of depth d_i therefore represents an interval of length 2^{-d_i}. At completion these intervals are disjoint, giving

$$\sum_i 2^{-d_i} \le 1, \qquad 2^{-\frac{1}{n}\sum_i d_i} \le \frac{1}{n}\sum_i 2^{-d_i} \le \frac{1}{n}.$$

Convexity gives the bound on $2^{-\frac{1}{n}\sum_i d_i}$. Hence $\sum_i d_i \ge n \log n$, and at least $\frac{1}{2} n \log n$ comparisons are necessary [4, Section 4.1].

The tree representation can settle relations that have never been queried. For example, start again with four items at the root and answer $A < B$ and $C < D$. Placing A, C in the left child and B, D in the right child also commits the adversary to $A < D$ and $C < B$. Neither relation follows from those two answers alone.

2.2 The adversary of Kaplan, Zamir, and Zwick

Kaplan, Zamir, and Zwick [6] replace each step of a four-child tree by a small directed acyclic graph (DAG), allowing an item to move towards a group of consecutive children before selecting one child. Number the children $0, 1, 2, 3$. The intermediate node $[i, j]$ represents the possible destinations $i, i+1, \ldots, j$. The parent itself is $[0, 3]$, and $[i, i]$ is child i. For $i < j$, a downward step removes one endpoint of the range, reaching $[i, j-1]$ or $[i+1, j]$. Figure 2 shows this interval lattice. For example, $[0, 2]$ and $[1, 3]$ both lead to $[1, 2]$; the intermediate nodes have shared descendants. The same construction is repeated at every node of the four-child tree.

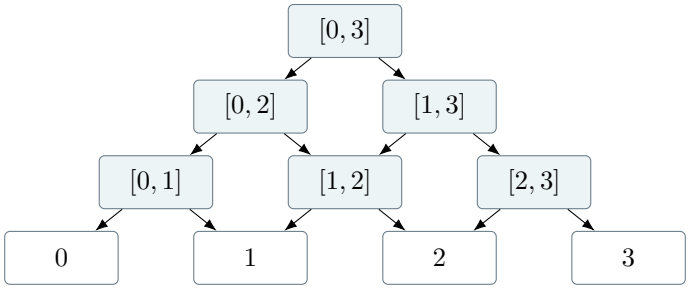


Figure 2: One step of the four-child tree in the adversary of Kaplan, Zamir, and Zwick [6, Section 5]. An intermediate node records a range of possible children. In particular, an item at $[1, 3]$ has not committed to child 1, 2, or 3.

Within a node, the adversary stores unpaired items and disjoint ordered pairs. There can be many items at a node, but its retained comparison components have at most two vertices. Tree nodes and intermediate nodes representing three children may store pairs; intermediate nodes representing two children store only unpaired items. Comparing two unpaired items A, B at a tree node can therefore record $A < B$ as a pair without moving either item.

A comparison between pairs in different nodes illustrates the role of the lattice. Suppose $A < B$ is retained at $[0, 2]$ and $C < D$ at $[1, 3]$. On comparing A and C, one permitted response

is $A < C$: move A into child 0 and B into $[1, 2]$, leaving the pair $C < D$ at $[1, 3]$. Figure 3 shows the change. The old answer $A < B$ is now represented by the items' positions, while $C < D$ remains a retained pair [6, Table 11].

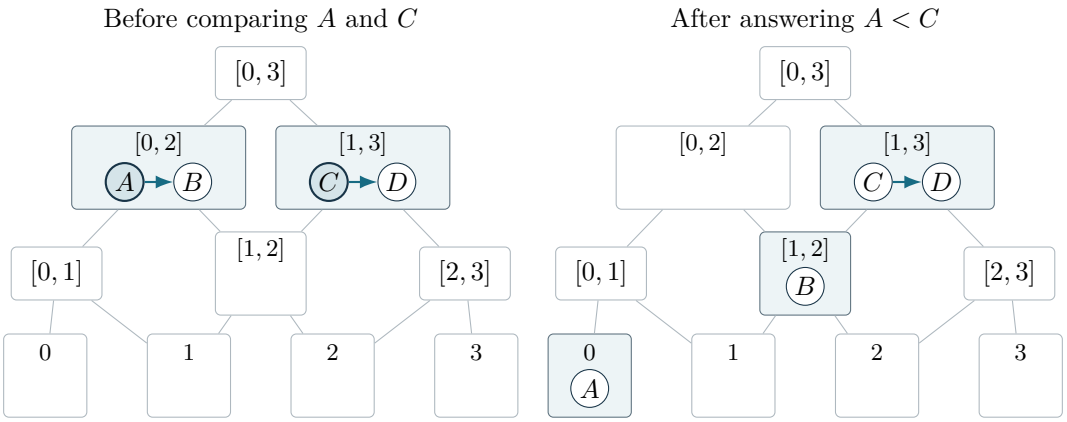


Figure 3: A comparison between pairs at different nodes in the adversary of Kaplan, Zamir, and Zwick [6, Table 11]. Shaded items are queried. Blue arcs are retained comparisons; gray lines are lattice edges. The new answer $A < C$ is enforced by the disjoint child ranges assigned to A and C.

A retained pair also restricts its members' possible destinations: they cannot enter the same child of the underlying four-child tree. For example, if $A < B$ is stored at $[0, 2]$, then A can enter only child 0 or 1, and B can enter only child 1 or 2. The pair's order thus narrows each member's possible destinations, and the adversary uses these restrictions when choosing subsequent comparison answers [6, Section 5].

Their analysis measures progress using item depths, intermediate placements, and retained pairs. The comparison rules bound the increase in this measure and give a lower bound of $\frac{37}{64} n \log n$ comparisons, improving the earlier $\frac{1}{2} n \log n$ bound. Their concluding discussion proposes allowing more complicated comparison components than pairs, and asks whether an efficient deterministic adversary can force $(1 - o(1))n \log n$ comparisons [6, Section 6].

3 The adversary

We describe the data structure maintained by our adversary and how it is updated in response to comparison queries from the sorting algorithm.

Kaplan, Zamir, and Zwick [6] retain disjoint ordered pairs within a node. Our adversary maintains a rooted tree with a DAG at each node, so several retained comparisons can involve the same item. For example, it can keep $a < b$ and $a < c$ together while leaving the order of b and c undecided. This lets it represent richer partial orders before moving items further down the tree.

The computational challenge is to use these richer orders without having to approximately count the linear extensions of an entire DAG. We periodically simplify a node's DAG by removing vertices and moving their items into child nodes, creating children when needed. Constraints at the parent preserve the comparisons involving the moved items, while their order within a child can remain unresolved. These simplifications allow comparison answers to be chosen by deterministic calculations on small neighborhoods in that node's DAG. The analysis must show that the resulting loss of possible orders is small enough to retain the optimal comparison bound.

3.1 The data structure

Our data structure is a rooted tree. Each node stores item records and a single directed acyclic graph (DAG), whose vertices are the items recorded at that node. We call this its *in-node DAG*. Figure 4 shows an example.

Each item has a record (x, X_x, a_x, b_x) stored at exactly one node. The label x identifies its key, and X_x is a symbolic variable for its position among the items at that node. The fields a_x, b_x restrict this variable to the open interval $(a_x, b_x) \subseteq (0, 1)$. The bounds a_x, b_x are fractions whose denominators are powers of two; we call such numbers *dyadic rationals*. We call an item *free* at its node while its position remains variable.

In an in-node DAG, an arc $x \to y$ records a comparison answered $x < y$ between two items stored there, imposing $X_x < X_y$ on their positions. A feasible assignment of positions at that node satisfies both its interval bounds and its arc inequalities. Each answered comparison between free items contributes at most one arc.

Initially the tree consists of just the root, containing all records with bounds $a_x = 0$ and $b_x = 1$. The position variables X_x are unassigned, and the root's in-node DAG has no arcs. Thus every assignment in $(0, 1)^n$ is feasible, and every total order can be obtained by assigning distinct positions.

Each child is attached to a fixed position $c \in (0, 1)$ in its parent, called its *anchor*. A node has at most one child at each anchor. For any feasible assignment at the parent, an item with position below c precedes every item in that child's subtree; an item with position above c follows every item in it. Subtrees of different children are ordered by their anchors. All items of each child subtree therefore occur consecutively in the resulting total order. Within the child, position variables again range over $(0, 1)$, and its in-node DAG and interval bounds describe the possible orders inside this block.

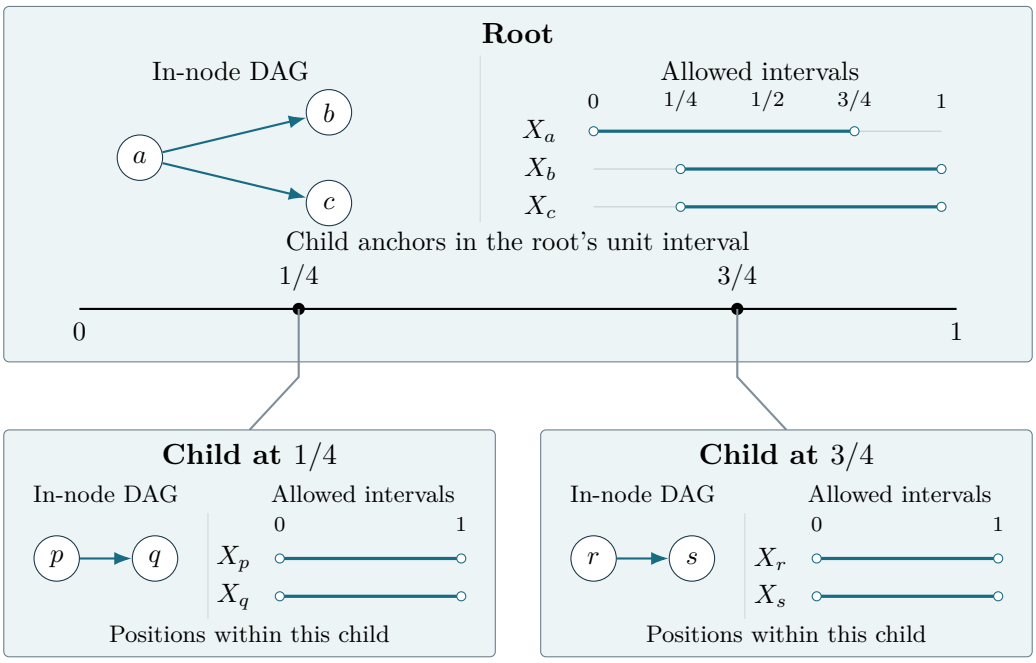


Figure 4: A state of the adversary. Each large box is a tree node, containing an in-node DAG and one open interval for each of its items.

In the root of Figure 4, the in-node DAG records $a < b$ and $a < c$. To read an order from this state, choose feasible positions at each node, avoiding ties with other items or child anchors. List the items and anchors from left to right, and recursively replace each anchor by its child's order.

For example, root positions $(X_a, X_b, X_c) = (1/3, 1/2, 7/8)$ give

$$p < q < a < b < r < s < c.$$

Other feasible choices give other orders. Every order represented this way must respect all previous answers to the sorter. These answers form the *global transcript*. We also maintain a *global comparison DAG* on the original item labels, inserting an arc $A \to B$ whenever the adversary answers $A < B$. An in-node DAG contains the comparisons between items currently stored at that node. Initially the global transcript is empty, and the global comparison DAG has all n labels as vertices and no arcs.

3.2 Answering a comparison

Algorithm 1 shows the algorithm the adversary uses to answer a comparison by the sorter.

To compare A and B, start at the root and descend into a child whenever its subtree contains both items. Stop when at least one item is stored at the current node, or when the items lie in different child subtrees. At this node, use an item's free coordinate if it is stored here, and the corresponding child anchor otherwise. In Figure 4, comparing p with q descends into the left child; comparing p with r uses the anchors $1/4$ and $3/4$; and comparing a with p compares X_a with $1/4$ at the root.

Two distinct anchors determine the answer immediately. In the other cases, first propagate the interval bounds through the in-node DAG. If there is an arc $y \to x$, every lower bound on X_y is also a lower bound on X_x; upper bounds propagate in the reverse direction. Write $y \preceq x$ when $y = x$ or a directed path leads from y to x. The resulting *effective interval* is

$$\ell_x = \max_{y \preceq x} a_y, \qquad r_x = \min_{y \succeq x} b_y, \qquad W_x = r_x - \ell_x. \tag{1}$$

The operation `Tighten` stores these bounds as a_x, b_x. It is performed after every change to its in-node DAG or intervals. Bounds remain stored even if the arcs that implied them are later removed. An interval bound or a directed path may already decide the comparison; in that case return the determined answer.

For an undecided comparison, let p be the probability that A precedes B when positions at this node are assigned uniformly subject to its interval bounds and in-node DAG inequalities. The subroutine `Estimate` returns a deterministic estimate $\widehat{p}$. Fix a rational error tolerance $0 < \eta < 1/4$. Choose $A < B$ if $\widehat{p} \geq 1/2$, and $B < A$ otherwise. For two free items, the event being estimated is $X_A < X_B$. For a free item x and an anchor c, estimate $X_x < c$, taking the complement if the queried order is reversed. An answer between two free items adds an arc to the in-node DAG; an answer involving an anchor adds an interval bound. Repeated comparisons never add duplicate arcs.

The estimator also requires a lower bound on the width of at least one queried free coordinate. Write this threshold as $1/R$, with rational $R \geq 1$. Call a coordinate *wide* when $W_x \geq 1/R$, and *narrow* otherwise. If all queried free coordinates are narrow, move one of their items into a child before trying the query again. Such a move is called a *refinement*, denoted `Refine`(x); Section 3.3 gives the operation in Algorithm 3.

The calculation also needs bounds on the in-node DAG. Fix positive integers k and H. We limit each vertex to fewer than k neighbors, counting both incoming and outgoing arcs, and each directed path to fewer than H vertices. After answering, refine vertices with at least k neighbors until none remain. Then select a path of H vertices and refine all of them, repeating until no such path remains. The procedure `Repair`(v) performs these *repairs* at node v; see Algorithm 2. A path repair includes the entire selected list, even if earlier moves break the path.

The following lemma supplies the accuracy needed by the comparison rule and the anchor search. Its degree and path bounds also cover intermediate states during a repair. Section 6 gives the estimator and proves the lemma.

Lemma 3.1 (Probability estimation)**.** *Suppose a node admits a feasible assignment, and its in-node DAG has degree at most k and directed paths of at most $2H$ vertices.* `Estimate` *deterministically computes, with error at most η:*

(i) $\mathbb{P}(X_x < c)$*, when x is wide and c is dyadic;*
(ii) $\mathbb{P}(X_x < X_y)$*, when at least one of x, y is wide.*

Probabilities use the uniform distribution on feasible assignments. The calculation terminates and leaves the stored state unchanged.

Algorithm 1: `Compare`(A, B): answer a comparison query.

Input: Distinct item labels A, B and the current adversary state.
Output: An answer $A < B$ or $B < A$, with the state updated.
while *true* **do**
  $v \leftarrow$ the root
  while *A and B lie in the same child subtree of v* **do**
    $v \leftarrow$ that child
  if *A and B lie in different child subtrees of v* **then**
    answer $\leftarrow$ the left-to-right order of their child anchors
    Append *answer* to the global transcript
    Insert its arc into the global comparison DAG
    return *answer*
  `Tighten`(v)
  if *interval bounds or a directed path in the in-node DAG at v determine the answer* **then**
    answer $\leftarrow$ the determined answer
    Append *answer* to the global transcript
    Insert its arc into the global comparison DAG
    return *answer*
  if *at least one queried free coordinate is wide* **then**
    break
  `Refine`(the selected queried free item)
$\widehat{p} \leftarrow$ `Estimate`$(v, A < B, \eta)$
answer $\leftarrow A < B$ if $\widehat{p} \geq 1/2$, and $B < A$ otherwise
Append *answer* to the global transcript
Insert its arc into the global comparison DAG
Store *answer* as an arc in the in-node DAG at v or as an interval bound
`Tighten`(v)
`Repair`(v)
return *answer*

Refining a queried item restarts the search from the root because the node that handles the query may change. Both `Compare` and `Repair` choose the smallest label among eligible items. `Repair` selects the lexicographically first eligible path and processes its items in label order. These rules make every selection deterministic.

Algorithm 2: `Repair`(v): restore the degree and path-length bounds.

Input: A tree node v and the current adversary state.
Output: The updated state, with degree below k and paths of fewer than H vertices in the in-node DAG at v.

```
while the in-node DAG at v has a vertex of degree at least k do
    Refine(such a vertex)
while the in-node DAG at v has a directed path on H vertices do
    P ← the selected path
    foreach x ∈ P in label order do
        Refine(x)
```

3.3 Moving an item into a child

Algorithm 3 gives `Refine`(x), which moves x from its current node into a child. After tightening the parent's interval bounds, it calls `ChooseAnchor` to select a position c inside x's effective interval.

The permitted anchor positions are obtained by repeatedly halving $[0, 1]$ and taking the midpoints of the resulting subintervals. Before any halving, the midpoint is $1/2$; after one halving, the two midpoints are $1/4$ and $3/4$. After m halvings there are 2^m equal subintervals, whose midpoints are

$$c = (i + \tfrac{1}{2})2^{-m}, \qquad i = 0, \ldots, 2^m - 1. \tag{2}$$

We call m the anchor's *level*. There are 2^m anchors at level m, and each anchor has a unique level. These positions are dyadic rationals, so using them as interval endpoints preserves the form of the stored bounds. Algorithm 4 in Section 5.3 gives the rule for selecting an anchor inside the effective interval.

Fixing $X_x = c$ places x in the child block anchored at c. The arcs incident to x in that in-node DAG become interval bounds: $y \to x$ gives $X_y < c$, and $x \to z$ gives $X_z > c$. Remove x and these arcs from the parent, retain the bounds, and propagate them through the remaining in-node DAG. Insert x into the child anchored at c, creating it if needed. In that child, x receives a fresh position variable with interval $(0, 1)$ and no incident arcs; the child's existing constraints remain unchanged. Figure 5 illustrates reuse of an existing child.

Algorithm 3: `Refine`(x): move an item into a child.

Input: A free item x and the current adversary state.
Output: The updated state, with x stored in a child of its former node.
$v \leftarrow$ the node storing x
`Tighten`(v)
$c \leftarrow$ `ChooseAnchor`(x)
foreach *arc* $y \to x$ *in the in-node DAG at* v **do**
  $b_y \leftarrow \min(b_y, c)$
foreach *arc* $x \to z$ *in the in-node DAG at* v **do**
  $a_z \leftarrow \max(a_z, c)$
Remove x's record from v and its vertex and incident arcs from the in-node DAG at v
`Tighten`(v)
if v *has no child anchored at* c **then**
  Create a child of v at c, with no records and an empty in-node DAG
$u \leftarrow$ the child of v anchored at c
Insert a record $(x, X_x, 0, 1)$ at u, with a fresh, unassigned position variable X_x
Insert x as an isolated vertex in the in-node DAG at u
`Tighten`(u)

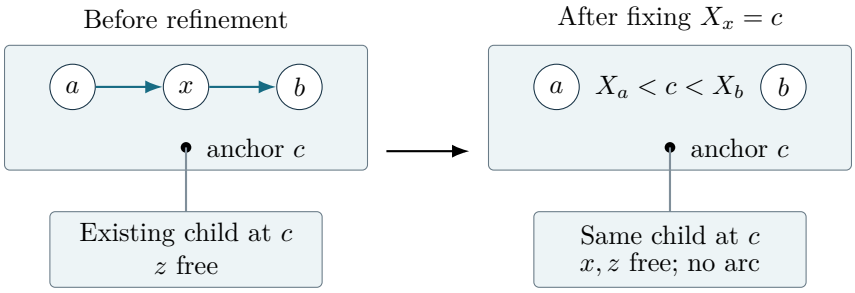


Figure 5: The two arc inequalities become bounds on the remaining parent coordinates. The new coordinate of x orders it within the child, where both $x < z$ and $z < x$ remain possible. The anchor c locates the child block among the parent items; the child's constraints determine the orders within that block.

Thus the parent fixes where the child block occurs, while the order of x within that block stays open. Lemma 5.2 proves that this operation preserves all answered comparisons.

Refinement can impose additional order relations. For example, suppose the root contains A, B, C with only $A < B$ recorded. Refining A at $1/2$ restricts B to $(1/2, 1)$. Refining C into the same child then leaves both $A < C < B$ and $C < A < B$ possible, while fixing the additional relation $C < B$. The comparison-bound proof accounts for these restrictions.

All thresholds are fixed before the first query. We choose their values where the proof needs them; Table 1 collects the values, their roles, and the sections introducing them. Section 4 proves the comparison bound using Lemmas 3.1 and 4.2. Sections 5 and 6 prove these lemmas, and Section 7 proves polynomial running time.

4 Why the adversary forces many comparisons

Recall from Algorithm 2 that a larger degree threshold k lets the adversary retain more comparisons involving each item before moving it into a child. Increasing k lets the adversary force a comparison count closer to $n \log n$. The following bound quantifies this dependence.

Theorem 4.1 (Comparison bound)**.** *For every power of two $k \geq 64$, the adversary of Algorithm 1, with the remaining parameter choices in Table 1, is deterministic and legal, and every call terminates. Every correct sorter makes at least*

$$\frac{n \log n - 3n}{1 + (2 \log k + 22)/k} \tag{3}$$

comparisons.

The coefficient of $n \log n$ tends to one as k increases. Section 7 chooses k as a function of n so that it grows without bound while the adversary's running time remains polynomial. For this section, fix a power of two $k \geq 64$.

The proof rests on two essential guarantees. Lemma 3.1 controls the error when choosing a comparison answer. Lemma 4.2 (Anchor selection) controls the restrictions imposed when moving an item into a child.

The proof uses a potential that starts at $n!$ and is at most 2^n when sorting finishes. The parameter choices make the maximum decrease in its logarithm per comparison arbitrarily close to one. Completed repairs increase it, so comparisons must account for a total decrease of nearly $\log(n!)$.

4.1 Feasible regions and anchor selection

To state Lemma 4.2 (Anchor selection), we first define the feasible region at a node and the density of an item's position.

At a node v, the stored coordinates range over

$$K_v = \{X : a_x < X_x < b_x \text{ for every free } x, \quad X_x < X_y \text{ for every in-node arc } x \to y\}. \tag{4}$$

Write $V_v = \mathrm{vol}(K_v)$. Under the uniform distribution on K_v, the probability of an event is the volume of the assignments satisfying it, divided by V_v. For example,

$$\mathbb{P}(X_x < X_y) = \frac{\mathrm{vol}(K_v \cap \{X_x < X_y\})}{V_v}.$$

The coordinates stored at different nodes are separate variables. Their feasible regions form a Cartesian product of total dimension n, with volume

$$Z = \prod_v V_v. \tag{5}$$

A node with no free items contributes volume one. Initially $Z = 1$; whenever every node has positive volume, $0 < Z \leq 1$.

Fixing one coordinate $X_x = c$ leaves a section of K_v. Let $S_x(c)$ be its volume in the remaining coordinates. The density of X_x is

$$f_x(c) = \frac{S_x(c)}{V_v}, \qquad \mathbb{P}(a < X_x < b) = \int_a^b f_x(t)\, dt.$$

Refinement replaces X_x by a fresh unit coordinate in a child, so the volume change is $Z_{\text{after}}/Z_{\text{before}} = f_x(c)$ (Lemma 5.3). This is why the choice of anchor matters.

Lemma 3.1 makes each selected answer retain at least $1/2 - \eta$ of its node's volume: a report at least $1/2$ implies $p \geq 1/2 - \eta$, and a smaller report implies $1 - p > 1/2 - \eta$. For refinement we need a lower bound on $f_x(c)$ in terms of the anchor's level. Lemma 4.2 gives this bound.

Set

$$B = (k+1)^{2H+1}. \tag{6}$$

In an in-node DAG of degree at most k and directed paths of at most $2H$ vertices, every coordinate comparable with x lies within undirected distance $2H$ of x. Thus B bounds the number of other coordinates comparable with x.

Lemma 4.2 (Anchor selection)**.** *Suppose a node's feasible region has positive volume, its interval endpoints are dyadic, and its in-node DAG has degree at most k and directed paths of at most $2H$ vertices. For $R = 2^{2kB}$ and rational $0 < \eta < 1/32$,* `ChooseAnchor` *deterministically terminates for every free item x and returns a dyadic anchor $c \in (\ell_x, r_x)$ of level m satisfying*

$$\log f_x(c) \geq (1 - 1/k)m - 4. \tag{7}$$

If x is narrow, its level also satisfies

$$m > 2kB + 2. \tag{8}$$

The calculation leaves the stored state unchanged.

Section 5.3.3 proves Lemma 4.2 and explains the choice of R. We will also use the structural lemmas in Section 5, which show that all answers remain consistent and that comparable items cannot enter the same child.

4.2 Relating volume to the final order

To use the fact that sorting ends with a single total order, we must account for the order fixed by the tree. The product Z measures freedom within individual nodes. For example, two items stored alone in separate children can each retain the full interval $(0, 1)$, giving $Z = 1$, even though the child anchors fix their order.

We therefore place all items on a common line, with coordinates whose order agrees with the total order represented by the tree. Each child subtree receives a small interval on its parent's line, and its positions are scaled to fit inside that interval. The construction is recursive; Section 5.5 gives the precise map and proves that it preserves order. The weights below record how much this construction shrinks volume.

For a child at an anchor of level m, reserve an interval of width

$$w(m) = \frac{2^{-(1+1/k)m}}{4k}. \tag{9}$$

There are 2^m possible anchors at level m. The extra factor $2^{-m/k}$ makes the sum of the widths finite over all levels; the factor $4k$ leaves space for items stored in the parent.

Let s_c be the number of original labels in the subtree below a parent–child edge c, and let w_c be its reserved width. Define

$$\mathcal{W} = \prod_{\text{parent–child edges } c} w_c^{s_c}. \tag{10}$$

Each item contributes the product of the widths on its path from the root. A comparison leaves $\mathcal{W}$ unchanged. Refinement at level m multiplies it by $w(m)$: one item crosses that parent–child edge, and its membership in every other subtree stays unchanged. Initially $\mathcal{W} = 1$.

The construction also scales each item's own free coordinate by $1/2$ to make room for child intervals. These n factors, together with the scalings recorded in $\mathcal{W}$, give a region of volume $2^{-n}Z\mathcal{W}$ (Lemma 5.8). At termination, all its assignments must give the same total order and hence lie in one of the $n!$ equal-volume order regions of $(0, 1)^n$. Consequently,

$$n!Z\mathcal{W} \leq 2^n.$$

Thus $n!Z\mathcal{W}$ starts at $n!$ and is at most 2^n when sorting finishes. To use this decrease to count comparisons, we must also account for the changes made by refinements.

4.3 The potential function

Refinement reduces $\mathcal{W}$. Lemma 4.2 gives a compensating increase in Z when the anchor's level is large, and refining a vertex of high degree removes many in-node arcs. We give additional weight to these two effects so that every completed repair increases the potential.

For the calculations, write $j = \log k$, so $j \geq 6$ and $k = 2^j$. Let e count the arcs in all in-node DAGs, and put

$$s = \frac{j+10}{k}, \qquad \Psi = n!Z\mathcal{W}\,(Z2^{-e})^s = n!Z^{1+s}\mathcal{W}\,2^{-se}. \tag{11}$$

The exponent $1+s$ gives extra weight to changes in $\log Z$. Deleting r in-node arcs increases the factor 2^{-se} by 2^{sr}. These quantities are used in the proof; computing them is unnecessary for answering a query.

Initially $Z = \mathcal{W} = 1$ and $e = 0$, so $\Psi = n!$. At termination, Lemma 5.8, $Z \leq 1$, and $e \geq 0$ give

$$\Psi = (n!Z\mathcal{W})(Z2^{-e})^s \leq 2^n. \tag{12}$$

For $j \geq 6$, the coefficient satisfies $0 < s \leq 1/4$: its value at $j = 6$ is $1/4$, and increasing j decreases it.

Consider a refinement at level m that deletes r incident in-node arcs. Lemmas 5.3 and 4.2, together with the child width, give

Quantity	Change caused by the refinement
$\log Z$	at least $(1-1/k)m - 4$
$\log \mathcal{W}$	exactly $-(1+1/k)m - (j+2)$
e	exactly $-r$

The second row uses $\log(4k) = j+2$. Combining these changes gives the following bound, which we will apply to all three repair rules.

Lemma 4.3 (Refinement inequality)**.** *Set*

$$a = s - \frac{s+2}{k}, \qquad C_j = j + 6 + 4s. \tag{13}$$

A refinement at level m deleting r in-node arcs satisfies

$$\log \Psi_{\text{new}} - \log \Psi_{\text{old}} \geq am + sr - C_j. \tag{14}$$

Moreover, $a > 0$.

Proof. The child receives an isolated coordinate, so exactly the r incident arcs in the parent's in-node DAG are deleted. The change in $\log \Psi$ is at least

$$(1+s)\big((1-1/k)m - 4\big) - (1+1/k)m - (j+2) + sr.$$

Its coefficient of m is $s - (s+2)/k$, and its constant term is $-(j+6+4s)$. Finally, $ks = j + 10 \geq 16 > s+2$, so $a > 0$. □

4.4 Every complete repair increases the potential

A path repair moves its items to distinct child anchors. Since there are only 2^m anchors at level m, a long path forces many of these moves to use high levels. We choose the path threshold

$$H = 2^k. \tag{15}$$

Lemma 5.9 then gives a total level of at least $H(k-2)$, which is enough to make the whole batch increase the potential.

Proposition 4.4 (Repair monotonicity)**.** *Each query-preparation refinement, each degree refinement, and each complete path batch increases* Ψ*.*

Proof. A degree refinement deletes at least k in-node arcs. Since $m \geq 0$ and $a > 0$,

$$\Delta \log \Psi \geq sk - C_j = 4(1 - s) > 0.$$

A preparation refines a narrow item. Lemma 4.2 gives $m > 2kB + 2 > 2k$, and hence

$$\Delta \log \Psi > 2ka - C_j = j + 10 - 6s > 0.$$

For a path batch, let m_i be the level chosen at its ith refinement. There are $H = 2^k$ refinements. Lemma 5.9 gives $\sum_i m_i \geq H(k - 2)$. Summing (14) over the whole batch and dropping the nonnegative arc terms gives

$$\begin{aligned} \log \frac{\Psi_{\text{after}}}{\Psi_{\text{before}}} &\geq H\,(a(k-2) - C_j) \\ &= H\left(2 - 7s + \frac{2s+4}{k}\right) > H/4 > 0, \end{aligned}$$

where the last inequality uses $s \leq 1/4$. □

An individual refinement within a path batch may decrease the potential. The proposition applies to the completed batch, which is why the final loop of Algorithm 2 finishes its selected list before selecting another path or returning to `Compare`.

4.5 The decrease caused by a comparison

The endpoint bounds require a net decrease of at least $\log(n!) - n$ in $\log \Psi$. Complete repairs increase this quantity. We now bound the amount by which a single comparison can decrease it.

Choose the estimation error to be

$$\eta = \frac{1}{100k}. \tag{16}$$

This makes the volume loss per answer at most $1 + 1/k$ bits, as shown below.

Proof of Theorem 4.1. Lemmas 3.1 and 4.2 give deterministic termination and the required accuracy and anchor bounds. With these lemmas, Proposition 5.7 establishes legality, and Lemmas 5.6 and 5.5 establish termination of every comparison call. All procedures and tie rules are deterministic.

Suppose the sorter terminates after q calls. Put $a_\eta = -\log(1/2 - \eta)$. By Lemma 3.1, an estimated answer retains at least $1/2 - \eta$ of its node volume, leaves $\mathcal{W}$ unchanged, and adds at most one arc to the affected in-node DAG. Its decrease in $\log \Psi$ is therefore at most

$$A_j = (1 + s)a_\eta + s.$$

A directly determined answer leaves the represented region and the potential unchanged. Preparations and complete restorations increase it by Proposition 4.4. Summing over calls and using the final bound (12) yields

$$\log(n!) \leq A_j q + n. \tag{17}$$

For $0 \leq u \leq 1$, convexity gives $2^{-u} \leq 1 - u/2$. Taking $u = 1/k$ shows

$$2^{-1/k} \leq 1 - \frac{1}{2k} \leq 1 - \frac{1}{50k}.$$

Since $\eta = 1/(100k)$, it follows that $a_\eta \le 1 + 1/k$. Consequently

$$A_j \le 1 + 2s + \frac{1+s}{k} < 1 + \frac{2j+22}{2^j}. \tag{18}$$

Finally,

$$\log(n!) \ge \int_1^n \log x \, dx = n \log n - \frac{n}{\ln 2} + \frac{1}{\ln 2}.$$

Combining this estimate with (17), (18), and $1 + 1/\ln 2 < 3$ gives $n \log n \le [1 + (2j+22)/2^j]q + 3n$. Substitute $j = \log k$ and rearrange to obtain (3). □

The following sections supply the deferred proofs. Section 5 proves Lemma 4.2 and the structural lemmas used above. Section 6 proves Lemma 3.1 by implementing the probability estimate. Section 7 then lets k grow with n while keeping the running time polynomial.

5 Refinement and its guarantees

We now justify the refinement claims used in Section 4. First, any interior anchor preserves consistency with previous answers. Next we give `ChooseAnchor` and prove Lemma 4.2 using the probability estimates from Lemma 3.1. We then establish the in-node DAG bounds, legality of the full procedure, and the geometric bound at termination.

5.1 Preserving the represented orders

We use the interpretation of the state given in Section 3. Assignments with equal free coordinates, or with a free coordinate equal to an anchor, lie on a finite union of sets of volume zero and are excluded when reading an order. We maintain the following invariant:

> Every represented total order respects every comparison already answered.

The state may discard some orders that still respect the answers. Legality requires only that at least one remains. Positive volume in every node will ensure this.

The global comparison DAG records every answered relation. The invariant must continue to hold when refinement removes an arc from an in-node DAG.

Lemma 5.1 (Effective intervals and sections)**.** *For an in-node DAG, tightening leaves the feasible region unchanged. The region has positive volume if and only if every effective width is positive. Moreover, fixing $X_x = c$ at any $c \in (\ell_x, r_x)$ leaves a positive-volume section in the remaining coordinates.*

Proof. If $y \ne x$ precedes x on a directed path, every feasible assignment satisfies $X_x > X_y > a_y$; the stored bound $X_x > a_x$ covers $y = x$. Taking the maximum over these bounds gives $X_x > \ell_x$. The upper-bound argument is the reverse. Thus all bounds introduced by (1) already hold, so tightening changes no feasible assignment.

Positive volume requires positive widths. Conversely, suppose all widths are positive. Along an arc $x \to y$, the definitions give $\ell_x \le \ell_y$ and $r_x \le r_y$. Choose numbers $0 < \theta_x < 1$ increasing in a topological order, and set

$$X_x = (1 - \theta_x)\ell_x + \theta_x r_x.$$

Increasing θ_x to θ_y strictly increases this value because $r_x > \ell_x$. Replacing its endpoints by the larger endpoints ℓ_y, r_y cannot decrease it. Hence $X_x < X_y$. All coordinates are inside their intervals and all arc inequalities hold strictly, so a sufficiently small open neighborhood of this assignment is feasible.

Now fix $X_x = c$ and delete its coordinate. The inequalities imposed by its incident arcs become upper bounds c at its in-neighbors and lower bounds c at its out-neighbors. Upper bounds propagate only backward along arcs, and lower bounds only forward. Since all previous effective bounds remain stored, the new effective intervals are therefore

$$\begin{cases} (\ell_y, \min(r_y, c)), & y \text{ precedes } x, \\ (\max(\ell_y, c), r_y), & y \text{ succeeds } x, \\ (\ell_y, r_y), & y \text{ is incomparable with } x. \end{cases}$$

The first two cases are disjoint because the in-node DAG is acyclic. A predecessor retains positive width: $r_y > \ell_y$ and $c > \ell_x \geq \ell_y$. A successor does too: $r_y > \ell_y$ and $c < r_x \leq r_y$. All other widths stay positive. The criterion just proved, applied to the remaining in-node DAG, shows that the section has positive volume. □

Lemma 5.2 (Refinement and child admission)**.** *Suppose the node regions have positive volume and the order invariant holds. Refining x at an interior anchor c preserves both properties, including when the child at c already exists. The global transcript implies no relation between x and any label already in that child subtree.*

Proof. Consider first an existing child at c, and denote its label set by S. Choose a feasible assignment in the parent section $X_x = c$, with the other free coordinates pairwise distinct and different from every child anchor. Such an assignment exists by Lemma 5.1. Also choose feasible assignments in the other nodes.

There is a positive distance from c to every other child anchor and every free parent coordinate in this assignment. All incident inequalities at x are strict. Thus, with everything else fixed, x can be moved slightly to either side of c while keeping the old state feasible. The resulting two represented orders place x immediately before or immediately after the entire block S. Both orders respect the transcript. For each $y \in S$, one has $x < y$ and the other has $y < x$; neither relation can therefore be implied by the global comparison DAG.

Now take any order represented after the refinement. Deleting x from the child leaves an assignment of its old coordinates and hence an old child order. Apply the preceding perturbation using this assignment's parent coordinates and old child coordinates. It gives two feasible old orders that agree with the new one on every label except x. The new order inserts x somewhere into their child block. No answered comparison joins x to a label in S, so this insertion violates none of them. Relative to every label outside $S \cup \{x\}$, the whole block occupies the old anchor position. The item x is consequently on the same side of each outside label as in either of the two feasible old orders above. Comparisons between other labels are unchanged. The new order therefore respects the entire transcript.

If there was no child at c, the same argument uses an empty block: the new singleton child places x where its fixed parent coordinate stood. Finally, the remaining parent region is a positive-volume section, and the child receives an independent coordinate in $(0, 1)$. All node regions remain positive. □

5.2 The volume gained by refinement

Recall that $Z = \prod_v V_v$ and that $f_x(c) = S_x(c)/V_v$ is the density of X_x under the uniform distribution on its node's feasible region. Fixing $X_x = c$ removes one coordinate from the parent; placing x in a child adds an independent unit coordinate there.

Lemma 5.3 (Refinement identity)**.** *Refining x at an interior anchor c satisfies*

$$\frac{Z_{\text{after}}}{Z_{\text{before}}} = f_x(c). \tag{19}$$

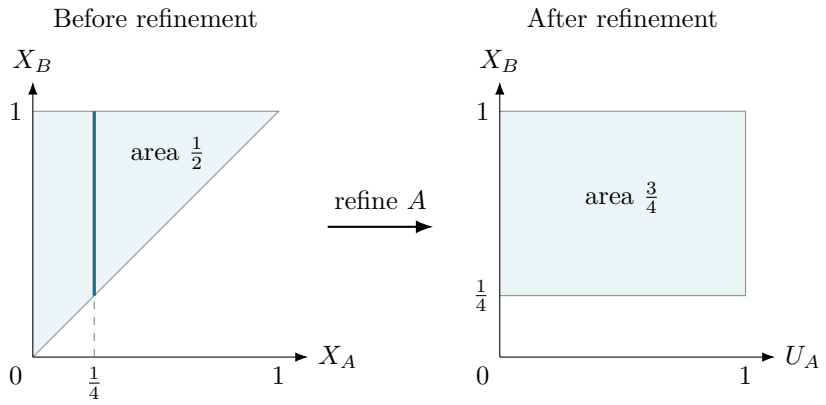


Figure 6: The highlighted section of the triangle has length 3/4. Replacing the fixed parent coordinate by a fresh unit coordinate in the child gives the rectangle on the right. Its area is 3/2 times the original area.

Proof. Let u be the child at c, taking $V_u = 1$ if it is newly created. The parent loses the coordinate X_x and retains precisely the section of volume $S_x(c)$. The child gains a new coordinate $U_x \in (0, 1)$, with no constraints involving its old coordinates. Its region becomes $K_u \times (0, 1)$ and still has volume V_u. Therefore the affected factors in the product change as follows:

$$V_v V_u \quad \longrightarrow \quad S_x(c) V_u.$$

Their ratio is $S_x(c)/V_v = f_x(c)$. All other node factors stay unchanged. The dimension lost in the parent is restored by the new child coordinate. □

For example, the parent region $0 < X_A < X_B < 1$ has area 1/2. Fixing $X_A = 1/4$ leaves the interval $X_B \in (1/4, 1)$ of length 3/4. The new child coordinate $U_A \in (0, 1)$ turns this into a rectangle of area 3/4, so refinement multiplies Z by 3/2; see Figure 6. Both regions represent the order $A < B$. Replacing the fixed parent coordinate by a fresh child coordinate gives the larger area.

5.3 Choosing an anchor

Consider a positive-volume node whose in-node DAG has degree at most k and directed paths of at most $2H$ vertices. Algorithm 4 gives `ChooseAnchor`. Lemma 4.2 requires a lower bound on the density at the chosen position. For a wide coordinate we find one by estimating its cumulative probability

$$F_x(t) = \mathbb{P}(X_x < t).$$

For a narrow coordinate the effective endpoints alone suffice. Each probability estimate uses Lemma 3.1.

Algorithm 4: `ChooseAnchor`(x): choose a position for refinement.

Input: A free item x stored at node v, its effective interval (ℓ_x, r_x), and the fixed parameters R, η.
Output: A dyadic anchor $c \in (\ell_x, r_x)$.

$W_x \leftarrow r_x - \ell_x$
if $W_x < 1/R$ **then**
  Choose the integer m with $W_x/8 < 2^{-m} \le W_x/4$
  $c \leftarrow$ the level-m anchor nearest to $(\ell_x + r_x)/2$, taking the smaller one on a tie
  return c
$a \leftarrow 0$; $b \leftarrow 1$
while *true* **do**
  $c \leftarrow (a+b)/2$
  $\widehat{p} \leftarrow$ `Estimate`$(v, X_x < c, \eta)$
  if $\widehat{p} < 1/4$ **then**
    $a \leftarrow c$
  else if $\widehat{p} > 3/4$ **then**
    $b \leftarrow c$
  else
    return c

For a wide item, each rejected midpoint halves the search interval $[a, b]$. The accepted interval's length is 2^{-m}, where m is the anchor's level. Both branches leave the stored state unchanged.

5.3.1 Density estimates used by the two rules

Suppose x has effective interval (ℓ, r) of width W and at most B other comparable labels. Then

$$f_x(t) \le \frac{B+1}{W}, \qquad f_x(t) \ge \frac{4^{-B}}{W} \quad \left(\ell + \frac{W}{4} \le t \le r - \frac{W}{4}\right). \tag{20}$$

The lower bound gives an endpoint-only choice: every point in the central half has this much density. The upper bound limits the probability in a short interval; it will ensure that the midpoint search cannot continue halving its search interval indefinitely.

For the search we also need a density bound that improves as its search interval shrinks. For any $a \le c \le b$,

$$(b-a) f_x(c) \ge \min\{F_x(c), 1 - F_x(c)\}(F_x(b) - F_x(a)). \tag{21}$$

If both tails at c and the mass of $[a, b]$ are bounded below, this forces the density at c to be a constant times $1/(b-a)$. Appendix A proves (20) and (21) by explicit maps between sections.

5.3.2 The midpoint search

Every test uses the same feasible region K and distribution F_x. Each test calls `Estimate` for $X_x < c$ with the same tightened endpoints. Only the queried threshold changes.

Lemma 5.4 (Midpoint search)**.** *The midpoint search for a wide item terminates at an interior anchor c. If its accepted search interval has length 2^{-m}, then*

$$f_x(c) \ge 2^{m-4}.$$

The search makes at most $\lceil \log(4(B+1)/W_x) \rceil + 1$ tests.

Proof. Every estimate has error at most $\eta < 1/32$. A test reported below $1/4$ has true value below $9/32$, and its midpoint becomes the new left endpoint. A test reported above $3/4$ has true value above $23/32$, and its midpoint becomes the new right endpoint. Since the initial endpoint values are $F_x(0) = 0$ and $F_x(1) = 1$, the search maintains

$$F_x(a) < 9/32, \qquad F_x(b) > 23/32.$$

At acceptance, the report lies in $[1/4, 3/4]$, so

$$\frac{7}{32} \le F_x(c) \le \frac{25}{32}, \qquad F_x(b) - F_x(a) \ge \frac{7}{16}.$$

The first bounds put c strictly inside its support. Applying (21) to the accepted search interval gives

$$2^{-m} f_x(c) \ge \frac{7}{32}\frac{7}{16} = \frac{49}{512} > \frac{1}{16},$$

which proves the density guarantee.

For termination, the upper density bound in (20) gives, at every search interval reached,

$$\frac{7}{16} \le F_x(b) - F_x(a) \le \frac{B+1}{W_x}\, 2^{-m}.$$

At $m \ge \lceil \log(4(B+1)/W_x) \rceil$, the upper bound would be at most $1/4 < 7/16$. Starting at $m = 0$ and increasing it by one at each rejection therefore permits at most the stated number of tests. □

5.3.3 The anchor guarantee

The wide-item search gives $\log f_x(c) \ge m - 4$. For a narrow item, the central density estimate gives the weaker bound $m - 3 - 2B$. To absorb this extra loss into the allowed fraction m/k of the level, choose

$$R = 2^{2kB}. \tag{22}$$

The rule $2^{-m} \le W_x/4 < 1/(4R)$ then forces $m > 2kB + 2$. In particular $2B < m/k$, so both anchor rules satisfy the required density bound.

Proof of Lemma 4.2. The in-node DAG bounds imply that x has at most B other comparable coordinates, by (6). For a wide item, Lemma 5.4 gives the stronger lower bound $m - 4$. For a narrow item, write $b = 2^{-m}$ for the spacing between consecutive level-m anchors, so $W_x/8 < b \le W_x/4$. The chosen midpoint is at distance at most $b/2 \le W_x/8$ from the interval center. It belongs to the central half and is therefore interior. By (20) and $W_x < 8b$,

$$f_x(c) \ge \frac{4^{-B}}{W_x} > 2^{m-3-2B}.$$

Moreover, $b \le W_x/4 < 1/(4R)$, so $m > \log R + 2 = 2kB + 2$. In particular $2B < m/k$, and hence $m - 3 - 2B > (1 - 1/k)m - 4$, proving (7). The midpoint search terminates by Lemma 5.4; the narrow rule makes a finite dyadic calculation. Both use deterministic tie rules and leave the stored state unchanged. □

The narrow rule must use its prescribed level even if an anchor of lower level lies inside the effective interval. Its high level is what allows the weaker central density estimate to satisfy the same bound as the wide-item search.

5.4 Bounds during a comparison call

We next check how the in-node DAGs evolve. The first two lemmas bound the changes during a call. We then combine them with Lemmas 3.1 and 4.2 to prove that every call preserves feasibility and consistency with the answered comparisons.

Lemma 5.5 (Query preparation)**.** *Before selecting its answer,* Compare *performs at most two refinements. Each selected coordinate is narrow, and preparation preserves the in-node DAG bounds.*

Proof. A preparation refines one of the two original queried items. Its new coordinate is isolated and has interval $(0, 1)$. Before an answer is stored, any further preparation only removes a parent coordinate and adds an isolated coordinate in a child. It cannot restrict this fresh coordinate. If the query descends into that child, the prepared item is therefore still wide. Each queried item can require preparation at most once. The selection test ensures narrowness. Deleting vertices and admitting isolated vertices preserve the degree and path-length bounds. □

Lemma 5.6 (In-node DAG bounds and termination)**.** *Suppose every in-node DAG has degree less than k and directed paths of fewer than H vertices at the start of a call. If its selected answer leaves positive volume, then every in-node DAG has degree at most k and directed paths of at most $2H$ vertices throughout the call. At its end, every in-node DAG again satisfies the strict bounds.* `Repair` *performs at most n refinements, all at the node changed by the answer.*

Proof. Preparation preserves the strict bounds by Lemma 5.5. The answer adds at most one arc to the affected in-node DAG, raising any degree there by at most one. Any new directed path consists of two old paths joined by that arc, so it has fewer than $2H$ vertices. A positive-volume answer cannot create a directed cycle. An interval change adds no in-node arcs.

Every refinement removes a vertex and its incident arcs from the parent. Thus degrees and path lengths cannot increase during restoration. The first loop of Algorithm 2 removes every degree violation. The second loop removes every path violation without increasing degrees. All selected path vertices are processed, even if earlier refinements have broken the original path. Each refinement removes a different free item from this node, so there are at most n. A child receives an isolated coordinate and keeps all its old arcs. Its degree and path-length bounds are preserved as well, so no child requires restoration. □

Consequently a complete call makes at most $n+2$ refinements. During the call, every coordinate has at most B other comparable coordinates in its in-node DAG: all lie within undirected distance $2H$ in an in-node DAG of degree at most k. These are the bounds used by the local estimator and the anchor rules.

Proposition 5.7 (Legality)**.** *Every reached state has positive volume and represents only orders consistent with the global transcript. After every complete call, all nodes satisfy the strict degree and path-length bounds.*

Proof. We induct on comparison calls and, within a call, on refinements. Initially the root region is the unit cube, every order is represented, and its in-node DAG has no arcs. Suppose the claims hold at the start of a call.

Preparation refines only narrow coordinates. Lemma 4.2 supplies an interior position, so Lemma 5.2 preserves positivity and the order invariant. Lemma 5.5 shows that preparation terminates with the width needed for a probability query, unless anchors, interval bounds, or a directed path already decide it.

Lemma 3.1 now applies to the positive queried region with the strict in-node DAG bounds. Its chosen answer retains at least $1/2 - \eta > 0$ of the volume and makes that comparison true

in every represented order. In particular, a new arc cannot create a directed cycle. An answer determined by anchors, interval bounds, or a directed path was already true in every represented order and leaves the state unchanged. The selected answer is recorded in the global comparison DAG before calling `Repair`.

Lemma 5.6 gives degree at most k and directed paths of at most $2H$ vertices during restoration. At each refinement these bounds and positivity allow Lemma 4.2 to be used. The chosen anchor is interior, and Lemma 5.2 preserves the invariant and positivity for the next step. Restoration terminates with the strict bounds. This completes the induction. The anchor estimates used at each step apply to arbitrary positive-volume node regions with the stated in-node DAG bounds. The proof of Lemma 3.1 in Section 6 applies to all such regions independently of the adversary's history. □

5.5 The geometric bound at termination

We now construct the common coordinates used in Section 4.2 and calculate their effect on volume. The map expands each child anchor into an interval and fits the child subtree recursively inside it. We prove that this map is injective and preserves the represented order, giving the required bound at termination.

5.5.1 Placing the children on a common line

Use the widths $w(m)$ defined in (9). Only children currently present receive intervals. The sum over all possible anchors below is a bound on every such finite set. The factor $4k$ leaves at least half the line for free positions. Indeed, at every node

$$\sum_c w_c \le \frac{1}{4k} \sum_{m \ge 0} 2^{-m/k} = \frac{1}{4k(1-2^{-1/k})} \le \frac{1}{2}. \tag{23}$$

For the last inequality, convexity gives $2^{-t} \le 1 - t/2$ for $0 \le t \le 1$, and we take $t = 1/k$.

At a node, map a free position t distinct from all anchors to

$$G(t) = \frac{t}{2} + \sum_{c<t} w_c. \tag{24}$$

The jump at an anchor c leaves the open interval

$$J_c = \left(\frac{c}{2} + \sum_{a<c} w_a, \frac{c}{2} + \sum_{a<c} w_a + w_c \right).$$

Place the recursively mapped child state inside J_c, using an affine map from $(0,1)$ to J_c. The intervals J_c are disjoint, occur in anchor order, and fit in $(0,1)$ by (23). Free positions stay in the gaps between them. All these widths are used only in the proof.

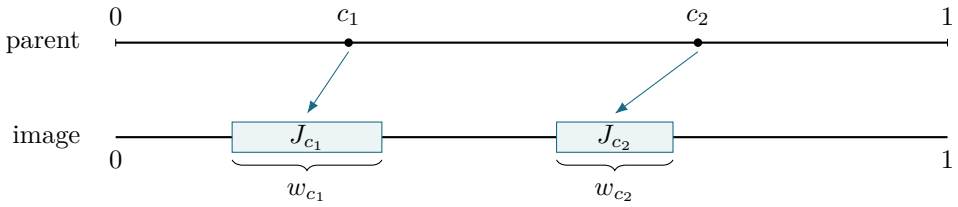


Figure 7: The packing map has slope 1/2 between anchors. Its jump at each anchor reserves an interval for the entire child subtree. The widths are shown schematically; their values are given by (9).

An item is scaled by a child interval's width every time its route from the root passes through that child. For example, an item at depth two has final position

$$a_1 + w_1\big(a_2 + w_2 G_v(X_x)\big),$$

where a_i is the left endpoint of the chosen child interval, measured in its own parent's unit interval, and G_v is the map in the group where the item is free. Its scale factor is $w_1 w_2/2$. The factor $1/2$ occurs once, at its free coordinate; the ancestor groups contribute only their child widths.

An edge width w_c scales the s_c labels in its subtree. Multiplying these factors over all items gives exactly $\mathcal{W}$ from (10), together with one factor $1/2$ for each free coordinate.

Lemma 5.8 (Packing volume)**.** *The recursive map is injective and preserves the represented order. Its image $\mathcal{I} \subseteq (0,1)^n$ has volume*

$$\mathrm{vol}(\mathcal{I}) = 2^{-n} Z\mathcal{W}. \tag{25}$$

Consequently, if the transcript determines a unique total order, then

$$n! Z\mathcal{W} \le 2^n. \tag{26}$$

Proof. The map G puts each free position on the correct side of every child interval. Recursion therefore preserves the order of the original labels. Anchors and widths are fixed within the state. The image coordinate of a label depends only on its one free coordinate, through a strictly increasing map, so the whole map is injective.

Partition the domain at the finitely many anchor hyperplanes. On each piece, the map of a label is affine. Its slope is $1/2$ at its free coordinate, multiplied by the widths of the child intervals containing that label along the path to the root. There are exactly n factors of $1/2$, one per free coordinate. An edge width w_c occurs s_c times. Multiplying these coordinate scale factors shows that every piece has its volume multiplied by $2^{-n}\mathcal{W}$. The domain, a product of the node regions, has volume Z. Injectivity lets us add the image volumes of the pieces, proving (25).

At completion, legality places the image in the region of the cube realizing the unique transcript order. The $n!$ coordinate-order regions have equal volume $1/n!$, since coordinate permutations preserve volume and equality hyperplanes have volume zero. Hence $2^{-n} Z\mathcal{W} \le 1/n!$. □

5.6 Why a long path must use fine anchors

The items on a directed path in an in-node DAG have already been compared, directly or through intermediate items. Two of them therefore cannot enter the same child. A long path needs many distinct anchors, but there are few anchors at low levels. The following estimate makes this observation quantitative. This is the path-level bound used in Proposition 4.4.

Lemma 5.9 (Total level of a path batch)**.** *The H labels of a selected path in an in-node DAG enter distinct children of their parent. If their levels are $m_1, \dots, m_H$, then*

$$\sum_{i=1}^{H} m_i \ge H(k-2). \tag{27}$$

Proof. Every in-node arc on the selected path records an answered comparison. The path's labels are therefore pairwise comparable in the global comparison DAG, and remain so after the in-node arcs are deleted. By Lemma 5.2, two of these labels cannot be admitted to the same child. The procedure completes all $H = 2^k$ refinements in this parent, so it uses H distinct anchors.

There are exactly 2^m possible anchors of level m. The smallest possible total level therefore uses all anchors of levels $0, \dots, k-1$ and one anchor of level k. Its value is

$$\sum_{m=0}^{k-1} m2^m + k = (k-2)2^k + k + 2 \geq H(k-2).$$

Every other choice of H distinct anchors has at least this total level. □

6 Computing comparison probabilities

We now implement `Estimate` and prove Lemma 3.1. The same subroutine chooses comparison answers and guides the wide-item anchor search. It averages the outcomes of a finite collection of coordinate updates, using exact dyadic arithmetic. The proof applies to any positive-volume region of the stated form, independently of how the adversary reached it.

6.1 Estimating a probability by a finite average

Fix the node handling the calculation, and let K be its region of feasible positions. The requested event is either $X_x < c$ for a dyadic threshold c, or $X_x < X_y$. First compute the effective endpoints using the full in-node DAG, keeping these bounds in temporary storage. The calculation uses a temporary vector Y of dyadic numbers, initialized at the effective lower endpoints. It visits the vertices in a fixed topological order, updating one number at a time. We call one such pass a *sweep*.

At the visit to x, compute the interval permitted by its current neighbors:

$$L_x = \max(\{\ell_x\} \cup \{Y_z : z \to x\}), \qquad U_x = \min(\{r_x\} \cup \{Y_z : x \to z\}). \tag{28}$$

Choose a number u from the finite set

$$\mathcal{U}_S = \left\{ \frac{1}{2S}, \frac{3}{2S}, \dots, \frac{2S-1}{2S} \right\}, \tag{29}$$

and replace Y_x by $(1-u)L_x + uU_x$. Here S is a power of two. The same update can be applied repeatedly, because the tightened lower endpoints satisfy all arc inequalities weakly, and every update preserves these weak inequalities. The interval $[L_x, U_x]$ can have length zero; the update then leaves Y_x at its sole permitted value.

The adversary enumerates every assignment of mesh values to a prescribed number of updates. Its estimate is the fraction of these computations whose final vector satisfies the requested event. This definition gives a deterministic calculation using maxima, minima, and dyadic arithmetic.

6.2 How much of the in-node DAG to retain

Suppose the full in-node DAG has maximum degree at most d and every directed path has at most h vertices. Degree counts neighbors without regard to arc direction. Fix the required error ε and the width bound $1/R$. The following choices specify all sizes in the finite calculation:

$$M = (d+1)^{h+1}, \qquad C = M + 1, \qquad p = (4M^2)^{h+1}, \tag{30}$$

$$\delta = \frac{\varepsilon}{32CR}, \qquad \lambda = \lceil \log(1/\delta) \rceil, \qquad \mu = \lceil \log(8/\varepsilon) \rceil, \tag{31}$$

$$t = p\lambda + \mu, \qquad \rho = ht. \tag{32}$$

Take S to be the least power of two at least $\max\{2, ht/(2\delta)\}$. These integers can be computed using rational arithmetic and comparisons with powers of two. The proof in Appendix B explains

their roles: t controls the number of sweeps, S controls the rounding error, and ρ includes every vertex that can influence a queried coordinate during those sweeps.

Retain all vertices within undirected distance ρ of a queried coordinate, together with all arcs between them. For a pair query, take the union of the two neighborhoods. Copy their effective endpoints from the full in-node DAG. These endpoints retain the interval restrictions implied by vertices outside the neighborhood. The retained subgraph has at most

$$b = 2(d+1)^{\rho+1} \tag{33}$$

vertices. Write b' for its actual number of vertices. Fix a topological order of this retained subgraph, choosing by item label whenever several vertices are eligible.

If a directed path already decides a pair query, its probability is zero or one. For a threshold outside the queried effective interval, the probability is likewise known. In the remaining cases use the following finite calculation. It leaves the adversary's stored state unchanged.

Algorithm 5: `Average`: estimate one comparison probability.

Input: The retained subgraph of the in-node DAG and its effective intervals, the integers t, S, and the requested event E.
Output: A rational estimate of the probability of E.

```
N_E ← 0
foreach assignment of mesh values to the b't updates do
    Y_x ← ℓ_x for every retained vertex x
    for i = 1, ..., t do
        foreach vertex x in the fixed topological order do
            Compute L_x, U_x from (28)
            u ← the mesh value assigned to this update
            Y_x ← (1 − u)L_x + uU_x
    if E holds for the final vector Y then
        N_E ← N_E + 1
return N_E / S^{b't}
```

Theorem 6.1 (Finite averaging)**.** *Let K have positive volume and be defined by intervals with dyadic endpoints and the arc inequalities of a DAG with degree at most $d \geq 1$ and directed paths of at most $h \geq 1$ vertices. Let $R \geq 1$ and $0 < \varepsilon < 1/4$ be rational. Use the globally tightened endpoints and the parameters above. Under the uniform distribution on K:*

1. *If $W_x \geq 1/R$, the estimate of $\mathbb{P}(X_x < c)$ has error at most ε for every dyadic threshold c.*
2. *If at least one of W_x, W_y is at least $1/R$, the estimate of $\mathbb{P}(X_x < X_y)$ has error at most ε.*

The calculation enumerates $S^{b't} \leq S^{bt}$ sequences of updates, each with $b't$ coordinate updates. Apart from finding the neighborhood and handling the input endpoints, its cost depends only on d, h, R, ε.

The proof compares the finite calculation with an assignment uniformly distributed on the full region. Updating a coordinate by a uniform point in its currently permitted interval preserves that distribution. When both this assignment and a vector started at the lower endpoints use the same update choices, their queried coordinates approach one another. Rounding the choices to the finite mesh adds a controlled error. After t sweeps the queried values depend only on vertices within distance ht, which justifies restricting the calculation to the retained subgraph.

The width assumption controls the probability of an almost tied comparison. If x has at most M other comparable vertices, and y is incomparable with it, Corollary A.2 gives

$$\mathbb{P}(|X_x - X_y| \leq a) \leq \frac{2(M+1)a}{W_x}.$$

The corresponding bound also holds for a fixed threshold. Thus a wide queried coordinate converts a bound on coordinate error into a bound on comparison error. Appendix B proves the theorem, and Appendix C bounds the exact arithmetic.

Finite computations can end with equal coordinates. For instance, one sweep on two independent unit intervals with $S = 4$ gives frequencies $3/8$ for each strict order and $1/4$ for equality. Accordingly, the comparison rule uses the estimate $\widehat{p}$ for one strict event and $1-\widehat{p}$ for the opposite answer. Since the continuous probabilities sum to one, this preserves the accuracy guarantee for both answers.

Proof of Lemma 3.1. The interval and arc inequalities are strict, so a feasible assignment has an open neighborhood of feasible assignments. The region therefore has positive volume. Computing the effective endpoints in temporary storage preserves this region by Lemma 5.1. Apply Theorem 6.1 with $d = k$, $h = 2H$, and $\varepsilon = \eta$ to obtain the stated accuracy for both kinds of event. The calculation enumerates finitely many choices in a fixed order and uses exact arithmetic, so it terminates deterministically. It uses copies of the stored constraints and leaves the adversary's state unchanged. □

The next section chooses k as a function of n and bounds the running time uniformly.

7 A uniform polynomial-time implementation

For each fixed j, the estimator uses a finite number of coordinate updates and mesh choices. To obtain the leading constant one, we must let j grow with n while keeping this work polynomial. We bound both the computation of the parameters and the size of every subsequent finite average. Computation is measured on a fixed deterministic multitape Turing machine, with integers represented in binary.

7.1 Selecting the parameter

For $j = 6, 7, \ldots$, set $k = 2^j$ and compute the remaining adversary parameters in Table 1 and the estimator parameters in (30)–(33), using $d = k$, $h = 2H$, and $\varepsilon = \eta$. Write t_j, S_j, ρ_j, b_j for the resulting values, and put

$$E_j = S_j^{b_j t_j}.$$

Here b_j bounds the size of every query neighborhood, so E_j bounds the number of update sequences for every estimate made with parameter j.

Fix a program that writes these complete parameter tuples successively, including E_j, in ordinary binary notation with delimiters. On input size n, simulate it for n machine steps. Among its completed tuples, choose the largest j satisfying

$$E_j \le n. \tag{34}$$

Use that parameter for the entire interaction. If no tuple qualifies, answer according to the fixed label order. The simulation writes only $O(n)$ bits; parsing the completed tuples and checking the inequality therefore take polynomial time as well.

These restrictions allow the selected parameter to grow without bound. For each fixed j, let T_j be the finite time required to finish its tuple. Whenever $n \ge \max\{T_j, E_j\}$, that tuple is available and qualifies. The selected $j(n)$ is then at least j. Thus $j(n) \to \infty$.

7.2 The cost of a finite average

For a chosen tuple, abbreviate its parameters by b, t, S. The bound (34) gives

$$bt \log S \le \log n. \tag{35}$$

Since $S \ge 2$, every enumerated sequence contains at most $bt \le \log n$ updates. Scanning the retained neighbors to form each update interval costs a polynomial in $\log n$ arithmetic operations. There are at most n sequences to enumerate.

If the input endpoints and threshold have at most Λ fractional binary digits, Appendix C bounds the extra precision during a sequence by

$$bt \log(2S) \le 2bt \log S \le 2 \log n.$$

The event tally has $O(\log n)$ bits. Thus the arithmetic cost of an estimate is polynomial in n and Λ, with an exponent independent of the chosen parameter.

7.3 Bounding the stored precision

Every stored endpoint is 0, 1, or a previously chosen dyadic anchor. For a wide coordinate, Lemma 5.4 bounds the number of midpoint tests and the precision of the chosen anchor by

$$K_0 = \lceil \log(4(B+1)R) \rceil + 2.$$

In this application the estimator has $M = B$ and $\delta = \eta/[32(B+1)R]$. Its choice $t \ge \lceil \log(1/\delta) \rceil$ implies $K_0 \le t + 2 = O(\log n)$. A wide refinement therefore makes $O(\log n)$ estimator calls and chooses an anchor with $O(\log n)$ fractional digits.

For a narrow coordinate, suppose all current endpoints have at most Λ fractional digits. A positive effective width is at least $2^{-\Lambda}$. The chosen spacing satisfies $2^{-m} > W_x/8 \ge 2^{-\Lambda-3}$, so $m < \Lambda + 3$. A level-m anchor has at most $m + 1 \le \Lambda + 3$ fractional digits. A narrow refinement therefore adds at most three digits. Tightening takes maxima and minima of stored endpoints and introduces no new numbers.

After P refinements, these facts give

$$\Lambda \le K_0 + 3P. \tag{36}$$

Lemmas 5.5 and 5.6 give $P \le (n+2)q$ after q complete calls, with at most $n + 2$ further refinements during the next call. Each refinement creates at most one node, so the tree has at most $P + 1$ nodes.

7.4 Bounding the state for every query sequence

A sorter may repeat comparisons indefinitely. After

$$Q_{\text{stop}} = 2n \lceil \log(n+1) \rceil \tag{37}$$

complete calls, stop using Algorithm 1. Choose a total order extending the comparison history by topologically sorting the global comparison DAG. Answer subsequent queries from the ranks in this fixed order and discard the recursive state, the stored transcript, and the global comparison DAG. The global comparison DAG records every previous answer, including directly determined answers, so the fixed order respects the full history.

The cutoff exceeds $n \log n$. A sorter reaching it has already made more comparisons than required by Theorem 4.1. Before the cutoff, the stored transcript has at most Q_{stop} entries, each

recording two labels and their order, so it uses $O(n \log^2(n+1))$ bits. Also $P = O(n^2 \log(n+1))$, and (36) gives

$$\Lambda = O(n^2 \log(n+1)).$$

The tree has polynomially many nodes. Every estimate enumerates at most n sequences, with $O(\log n)$ updates in each and operands of polynomial binary length. Ordinary integer addition, comparison, and multiplication therefore give a fixed polynomial bit bound for an estimate.

A call makes at most $n+2$ refinements. Each wide refinement makes $O(\log n)$ estimates; a narrow refinement uses only its endpoints. The call also makes at most one estimate to select its answer. All remaining operations are polynomial graph computations: routing the query, propagating bounds, finding neighborhoods, testing reachability, and selecting a long directed path in an in-node DAG. Path lengths are computed in reverse topological order. If H exceeds the number of vertices, no path qualifies; otherwise the selected path has at most n vertices. The fixed label rules can be implemented by these same computations. Thus every call, including the one that chooses the final fixed order, uses one fixed polynomial number of bit operations in n.

Proof of Theorem 1.1. The bounded initialization, finite averaging procedure, and cutoff define a single deterministic polynomial-time adversary. For sufficiently large n a tuple qualifies, and the chosen parameter satisfies $j(n) \to \infty$. If sorting finishes before the cutoff, Theorem 4.1 gives

$$q \geq \frac{n \log n - 3n}{1 + (2j(n) + 22)/2^{j(n)}} = (1 - o(1)) n \log n.$$

If the cutoff is reached, the sorter has already made more than $n \log n$ comparisons. Legality holds before the cutoff and is preserved by choosing a fixed order consistent with every answer. This proves the theorem. □

A Coordinate densities

We prove the density bounds used to choose anchors and to turn coordinate approximation into probability approximation. Let V be the set of coordinates, let $N = |V|$, and let $K \subseteq \mathbb{R}^V$ be their feasible region. For a coordinate x and a real number t, write

$$K_x(t) = \{(z_i)_{i \neq x} : z \in K,\ z_x = t\}, \qquad S_x(t) = \mathrm{vol}_{N-1} K_x(t).$$

Here $K_x(t)$ is the $(N-1)$-dimensional section obtained by fixing coordinate x at t. The uniform distribution on K therefore gives x the density

$$f_x(t) = \frac{S_x(t)}{\mathrm{vol}_N K}, \qquad F_x(t) = \int_{-\infty}^{t} f_x(s)\, ds.$$

These formulas define f_x pointwise, and we use this definition throughout. If some coordinates have already been fixed, we use the same definitions in the remaining free coordinates. In dimension zero, the volume of the singleton consisting of the empty assignment is one.

Lemma A.1 (Coordinate density bounds)**.** *Suppose x has at most B other comparable labels and effective interval (ℓ, r) of width W. Its density satisfies*

$$f_x(t) \leq \frac{B+1}{W}, \tag{38}$$

$$f_x(t) \geq \frac{4^{-B}}{W} \qquad (\ell + W/4 \leq t \leq r - W/4). \tag{39}$$

These bounds also hold after fixing any set of labels incomparable with x, for every resulting slice of positive volume.

Proof. We may fix any specified set of coordinates incomparable with x before starting the argument. Measure all volumes in the remaining free coordinates, and abbreviate the section volume by $S(t)$.

First we construct a map from the section at t into the section at a larger value s. Raising x may force its descendants to rise, but no other coordinates need move. We compress $[t, r]$ into $[s, r]$ and leave values above r unchanged. More precisely, for $\ell < t < s < r$, put $\alpha = (r-s)/(r-t)$ and apply

$$\phi(z) = \begin{cases} s + \alpha(z - t), & t \le z \le r, \\ z, & z \ge r \end{cases}$$

to x and all its descendants. Every descendant y has value greater than t. Its effective upper endpoint satisfies $r_y \ge r_x = r$, by the definition of effective intervals. A moved coordinate never decreases; if it starts below r, it stays below r, and if it starts at or above r, it stays fixed. Its lower and upper interval bounds are therefore preserved. The function ϕ is strictly increasing, so it preserves the inequalities imposed by arcs between moved coordinates. Every arc crossing the boundary of this set must enter it: an arc leaving a descendant would lead to another descendant. Only its head coordinate can move, and that coordinate never decreases, so its inequality also remains strict.

This injective map sends the section at t into the section at s. To compare volumes, partition the source section according to which moved coordinates lie below r. On each piece the map is affine and its Jacobian determinant in the free coordinates is α^j, where $j \le B$: the coordinate x is fixed, and at most B other coordinates are moved. Since $0 < \alpha < 1$, this determinant is at least α^B. The images of distinct pieces are disjoint because the map is injective. Adding their volumes gives

$$S(s) \ge \left(\frac{r-s}{r-t}\right)^B S(t) \qquad (t < s). \tag{40}$$

For a smaller target value s, the construction is reversed: lower x and its ancestors, compress $[\ell, t]$ into $[\ell, s]$, and leave values below ℓ fixed. Thus, for $\ell < s < t < r$, use

$$\phi(z) = \begin{cases} z, & z \le \ell, \\ s + \dfrac{s-\ell}{t-\ell}(z - t), & \ell \le z \le t. \end{cases}$$

on x and its ancestors. The effective lower endpoint of every ancestor is at most ℓ. No moved coordinate rises, and one that starts at or above ℓ remains there. Thus all intervals are preserved. The map is strictly increasing on the moved set, so it preserves the inequalities imposed by arcs within that set. Every arc crossing its boundary leaves the set. Only its tail coordinate can move, and that coordinate never increases, so these inequalities also remain strict. Its determinant on each piece is at least $((s-\ell)/(t-\ell))^B$. Consequently

$$S(s) \ge \left(\frac{s-\ell}{t-\ell}\right)^B S(t) \qquad (s < t). \tag{41}$$

Neither construction moves a coordinate incomparable with x, so both remain valid after the conditioning at the start of the proof. A region of positive volume has some section of positive volume. The two maps carry that section to every $t \in (\ell, r)$ with a positive volume factor. Consequently, after fixing incomparable coordinates, we normalize the density of x over the same effective interval (ℓ, r) of width W.

Normalize S to obtain f_x. Integrating the two inequalities on their respective sides of t gives

$$1 \ge f_x(t) \left(\int_\ell^t \left(\frac{s-\ell}{t-\ell}\right)^B ds + \int_t^r \left(\frac{r-s}{r-t}\right)^B ds \right) = \frac{W f_x(t)}{B+1}.$$

This proves the upper bound. For the lower bound, the density integrates to one over an interval of length W, so some $t_0 \in (\ell, r)$ has $f_x(t_0) \geq 1/W$. Fix $s \in [\ell + W/4, r - W/4]$. If $t_0 < s$, the ratio in (40) is at least $(r - s)/W \geq 1/4$; if $s < t_0$, the ratio in (41) is at least $(s - \ell)/W \geq 1/4$. If $s = t_0$, no map is needed. In every case $f_x(s) \geq 4^{-B}/W$. □

Corollary A.2 (A bound for small differences)**.** *If y is incomparable with x, or Y is a fixed number, then for every fixed interval I of length a,*

$$\mathbb{P}(X_x - Y \in I) \leq \frac{(B+1)a}{W}, \tag{42}$$

where $Y = X_y$ in the first case.

Proof. In the first case, condition on X_y. Lemma A.1 bounds the conditional density of x by $(B+1)/W$. Integrating over the translate $I + X_y$ and then averaging over X_y proves the claim. For fixed Y, apply the unconditional density bound. □

Lemma A.3 (Balancing sections)**.** *Let f be a coordinate density, extended by zero outside its effective interval. For $u \leq t \leq v$,*

$$f(u)f(v) \leq f(t)f(u + v - t). \tag{43}$$

Proof. Write $S(s) = \mathrm{vol}_{N-1} K_x(s)$. It suffices to construct a volume-preserving injection

$$K_x(u) \times K_x(v) \longrightarrow K_x(u + v - t) \times K_x(t).$$

Both products are measured in their $2(N-1)$ free coordinates. If $t = u$, $t = v$, or $S(u)S(v) = 0$, the inequality is immediate. Otherwise suppose $u < t < v$, take a pair of feasible assignments p, q with $p_x = u$ and $q_x = v$, and put $\delta = t - u$.

We first translate the second assignment by setting $h_i = q_i - \delta$ for every i. Translation preserves all arc inequalities, so both p and h satisfy them. The intermediate assignment h need not satisfy the interval bounds. We shall swap selected coordinates of p and h, then undo the translation on the second assignment.

Which coordinates must be swapped together? For each i, the two available values are p_i and h_i. Join the endpoints of an arc $i \to j$ by an undirected edge exactly when

$$\max(p_i, h_i) \geq \min(p_j, h_j).$$

An edge means that choosing the two endpoint values independently could violate $X_i < X_j$. If there is no edge, either value at i is strictly less than either value at j, so every choice is safe. Figure 8 illustrates the two cases.

Along an edge, $h_i - p_i$ and $h_j - p_j$ have the same strict sign. To see this, if $h_i \geq p_i$ and $h_j \leq p_j$, then $\max(p_i, h_i) = h_i < h_j = \min(p_j, h_j)$, because h satisfies the inequality for the arc $i \to j$. If $h_i \leq p_i$ and $h_j \geq p_j$, the same contradiction follows from $p_i < p_j$. These two cases also exclude a zero difference at either endpoint. Let C be the connected component containing x. Since $h_x - p_x = v - t > 0$, we obtain $h_i > p_i$ for every $i \in C$.

Now define the output pair by

$$(p'_i, q'_i) = \begin{cases} (q_i - \delta,\ p_i + \delta), & i \in C, \\ (p_i, q_i), & i \notin C. \end{cases}$$

This is precisely the proposed swap followed by the inverse translation. At x it gives $p'_x = v - \delta = u + v - t$ and $q'_x = u + \delta = t$, as required.

We check feasibility in two steps. First, on C we have $p_i < q_i - \delta < q_i$ and $p_i < p_i + \delta < q_i$. Thus both new values lie between the original feasible values p_i and q_i, and so remain in the

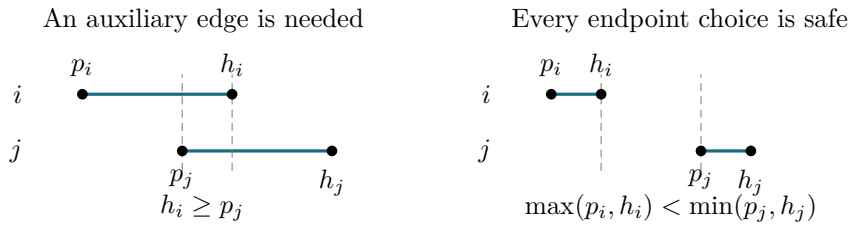


Figure 8: The auxiliary graph in Lemma A.3, for one arc $i \to j$. Each segment joins the two available values of its coordinate. On the left, swapping just one endpoint could select h_i and p_j, violating $X_i < X_j$. On the right, the two segments are separated in the required order, so swapping either endpoint is safe. Every boundary arc of a component has this second form.

interval for coordinate i. Outside C there is no change. Second, consider the arc inequalities before adding δ back. For an arc with both endpoints in C, the two assignments have exchanged values at both endpoints, so its inequality remains strict in each. If neither endpoint is in C, their values are unchanged. An arc crossing the boundary of C has no corresponding edge in the auxiliary graph, because C is a component; any choice of its endpoint values is therefore safe. Finally, adding the same δ to every coordinate of the second assignment preserves all arc inequalities. Both output assignments are feasible.

To recover the input, subtract δ from q' and observe that for every coordinate

$$\{p'_i, q'_i - \delta\} = \{p_i, q_i - \delta\} = \{p_i, h_i\}.$$

These unordered pairs determine the auxiliary graph. We therefore recover the same component C, swap its coordinates once more, and add δ to the second assignment. This recovers (p, q), proving injectivity.

There are only finitely many possible components C. On the part of the source with a fixed C, the map exchanges selected pairs of free coordinates and adds constants. Its Jacobian determinant has absolute value one. The parts and their images are measurable, and injectivity makes the images disjoint. Adding their volumes proves

$$S(u)S(v) \leq S(t)S(u + v - t).$$

Dividing by the square of the total volume proves the claim. □

Lemma A.4 (Density from mass on both sides)**.** *For a coordinate density f, its cumulative distribution F, and $a \leq c \leq b$,*

$$(b - a)f(c) \geq \min\{F(c), 1 - F(c)\}\big(F(b) - F(a)\big). \tag{44}$$

Proof. Lemma A.3 gives $f(u)f(v) \leq f(c)f(u + v - c)$ for $u \leq c \leq v$. For fixed $v \geq c$, integration over $u \leq c$ gives

$$F(c)f(v) \leq f(c) \int_{-\infty}^{c} f(u + v - c)\, du \leq f(c),$$

because the integral of a density over any interval is at most one. Integrating this inequality over $c \leq v \leq b$ yields

$$F(c)\big(F(b) - F(c)\big) \leq (b - c)f(c).$$

Likewise, for fixed $u \leq c$, integration over $v \geq c$ gives $f(u)(1 - F(c)) \leq f(c)$. Integrating over $a \leq u \leq c$ yields

$$\big(F(c) - F(a)\big)\big(1 - F(c)\big) \leq (c - a)f(c).$$

Add these inequalities and replace the two tail factors on the left by their minimum. □

B Accuracy of finite averaging

We prove Theorem 6.1 by comparing the finite calculation with coordinate updates on the full feasible region K. First we use continuous uniform choices to show convergence from the lower endpoints. Then we bound the effect of replacing these choices by mesh values and of retaining only a neighborhood of the query. Throughout, $d, h, M, C, p, \delta, t, \rho, S$ have the meanings given in Section 6.

B.1 Comparing two sequences of coordinate updates

Start X uniformly distributed on K. Conditional on all coordinates other than X_x, its possible values form the interval

$$\left(\max(\{\ell_x\} \cup \{X_u : u \to x\}), \min(\{r_x\} \cup \{X_v : x \to v\})\right).$$

The conditional density is constant. Replacing X_x by a uniform point in this interval therefore preserves the uniform distribution on K. This holds after every individual update and hence after every sweep.

Start a second vector Y at the tightened lower endpoints. Run both vectors on the full DAG in the same topological order, using the same fresh independent uniform number $U_x \in (0, 1)$ at each update. This joint construction is a coupling of the two sequences. The vector X remains uniformly distributed on K, while Y begins at a specified boundary point and approaches it under the shared updates.

Write $\Delta_x = |X_x - Y_x|$. Within a sweep, superscripts old and new mean the value at its start and after the coordinate's visit. Predecessors have already been updated, and successors retain their old values. The update rule therefore gives

$$\Delta_x^{\text{new}} \le (1 - U_x) \max_{u \to x} \Delta_u^{\text{new}} + U_x \max_{x \to v} \Delta_v^{\text{old}}, \tag{45}$$

where an empty maximum is zero. Indeed, the difference between the two lower endpoints is bounded by the largest predecessor discrepancy, and the difference between their upper endpoints by the largest successor discrepancy.

Let $B_h(x)$ be the vertices within undirected distance h of x. Starting at x, follow a predecessor attaining the maximum new discrepancy in (45), and repeat until reaching a source. This reverse path has $r \le h$ vertices. Every old successor value used along it belongs to $B_h(x)$. Put

$$D = \max_{z \in B_h(x)} \Delta_z^{\text{old}}.$$

At the source, the shared fixed lower endpoint contributes zero discrepancy. Substituting (45) backwards along the chosen path gives

$$\Delta_x^{\text{new}} \le \left(1 - \prod_{i=1}^{r} (1 - U_{z_i})\right) D.$$

Let A_x be the minimum of the fresh values $1 - U_z$ over x and all its ancestors. Each factor in this product is at least A_x, so

$$\Delta_x^{\text{new}} \le (1 - A_x^h) \max_{z \in B_h(x)} \Delta_z^{\text{old}}. \tag{46}$$

The maximizing path may depend on the fresh choices and discrepancies. Taking the minimum over the entire ancestor set makes the bound valid for every such path. Moreover, A_x depends only on fresh choices and is independent of the discrepancy vector at the start of the sweep.

B.2 A contraction bound independent of graph size

Both the ancestor set and $B_h(x)$ have at most $M = (d+1)^{h+1}$ vertices. Set $a_0 = 1/(4M^2)$. A union bound gives

$$\mathbb{P}(A_x < a_0) \le M a_0 = \frac{1}{4M}.$$

On the complementary event, the choice $p = (4M^2)^{h+1}$ gives

$$(1 - A_x^h)^p \le (1 - a_0^h)^p \le \frac{1}{1 + p a_0^h} = \frac{1}{1 + 4M^2} \le \frac{1}{4M}.$$

Here $(1-z)^{-p} \ge 1 + pz$ for $0 \le z < 1$ supplies the second inequality. Combining the two events yields $\mathbb{E}[(1 - A_x^h)^p] \le 1/(2M)$. Independence from the old discrepancies and (46) now imply

$$\begin{aligned} \mathbb{E}[(\Delta_x^{\text{new}})^p] &\le \frac{1}{2M} \sum_{z \in B_h(x)} \mathbb{E}[(\Delta_z^{\text{old}})^p] \\ &\le \frac{1}{2} \max_{z \in B_h(x)} \mathbb{E}[(\Delta_z^{\text{old}})^p]. \end{aligned} \tag{47}$$

Initially each discrepancy is at most one. Induction over sweeps gives $\mathbb{E}[\Delta_x(t)^p] \le 2^{-t}$ for every vertex. Thus

$$\mathbb{P}(\Delta_x(t) > \delta) \le 2^{-t} \delta^{-p} \le 2^{-\mu} \le \varepsilon/8, \tag{48}$$

using $t = p\lambda + \mu$, $2^\lambda \ge 1/\delta$, and $2^\mu \ge 8/\varepsilon$. The estimate holds separately at each queried vertex, with constants independent of the total number of vertices.

B.3 Replacing continuous choices by mesh values

Round each uniform choice U to the midpoint of its one of S equal subintervals of $(0,1)$. The rounded choices are independent and uniform on $\mathcal{U}_S$, and differ from the original choices by at most $1/(2S)$. Run a third vector $\widehat{Y}$ from the same lower endpoints, using these rounded choices on the full graph.

For a fixed current interval, changing the choice by at most $1/(2S)$ changes the updated coordinate by at most $1/(2S)$. Allowing the two current intervals to differ as well, let $e_x = |Y_x - \widehat{Y}_x|$. Then

$$e_x^{\text{new}} \le \max\left\{ \max_{u \to x} e_u^{\text{new}}, \max_{x \to v} e_v^{\text{old}} \right\} + \frac{1}{2S},$$

with zero discrepancy at fixed endpoints. Expanding through the newly updated predecessors uses at most h vertices. One sweep therefore adds at most $h/(2S)$ to the maximum old discrepancy. Since both vectors start at the same point, after t sweeps

$$|Y_x(t) - \widehat{Y}_x(t)| \le \frac{ht}{2S} \le \delta \tag{49}$$

for every vertex and every sequence of choices.

B.4 Restricting the calculation to the query neighborhood

Expand one update backwards through the new predecessor values used in its sweep. A directed path contains at most h vertices, so this expansion uses at most $h-1$ predecessor edges. It may then use one successor edge to read a value from the start of the sweep. Consequently, the final value at x depends on old values within distance h and fresh choices within distance $h-1$.

After t sweeps, only initial values, endpoints, and choices within distance ht can influence x. At distance exactly ht, only initial values are needed. Keeping this neighborhood therefore

reproduces the full rounded calculation exactly at x. For a pair query, keeping the union of both neighborhoods reproduces both values. The full graph's tightened endpoints are retained throughout, since they are inputs to every update.

One can use the restriction of a full topological order. Any other topological order of the retained graph gives the same result for fixed choices assigned to each vertex and sweep: updates of incomparable vertices commute, and adjacent exchanges of such vertices connect the topological orders. The neighborhood-size bound (33) follows from $1 + d + \cdots + d^\rho \le (d+1)^{\rho+1}$.

B.5 From coordinate error to comparison error

For an incomparable pair, choose a queried coordinate x with $W_x \ge 1/R$. There are at most M other vertices comparable with it. Corollary A.2 therefore gives

$$\mathbb{P}(|X_x - X_y| \le 4\delta) \le 8CR\delta = \varepsilon/4.$$

Outside the two exceptional events in (48), each continuous coordinate Y is within δ of its corresponding uniform coordinate X. Equation (49) adds at most another δ. If the comparisons made by X and $\widehat{Y}$ differ, either one of those exceptional events occurs or $|X_x - X_y| \le 4\delta$. Their total probability is at most $2(\varepsilon/8) + \varepsilon/4 = \varepsilon/2$.

For a threshold query there is one exceptional event. Otherwise, different answers require $|X_x - c| \le 2\delta$, which has probability at most $4CR\delta = \varepsilon/8$. The total disagreement probability is at most $\varepsilon/4$. A comparable pair is answered exactly by reachability, as specified in the estimator.

The difference between two event probabilities is bounded by the probability that their indicators disagree. The vector X remains uniform on K. Each possible sequence of rounded choices has probability $S^{-b't}$ on the retained graph, so its event probability is exactly the finite average in Algorithm 5. The preceding bounds prove Theorem 6.1.

C Exact arithmetic for the finite average

Let the retained graph have b' vertices, and suppose its dyadic endpoints and any query threshold have at most Λ fractional binary digits. A trajectory consists of $b't$ coordinate updates. Each update takes maxima and minima, followed by multiplication by mesh fractions with denominator $2S$. Charging a full denominator increase at every update bounds the precision of every intermediate coordinate by

$$\Lambda + b't \log(2S).$$

The values stay in $[0, 1]$, so their integer numerators have at most one additional bit. All comparisons and updates can therefore be carried out with exact integer arithmetic.

There are $S^{b't}$ trajectories. Enumerate their mesh choices as tuples of $b't$ integers in $\{0, \dots, S-1\}$. The event tally has at most $b't \log S + 1$ bits, and its denominator is the power of two $S^{b't}$. For a tally N and this denominator D, the comparison decision is $2N \ge D$. The midpoint search tests $4N < D$ and $4N > 3D$. These integer tests implement the probability rules exactly.

Section 7 chooses parameters satisfying $S^{bt} \le n$. Hence $b't \log S \le \log n$, the enumeration has at most n terms, and the extra coordinate precision is at most $2 \log n$ bits. These bounds, together with the bound on stored endpoint precision, give a polynomial bit cost for each estimate.

D Parameter reference

Table 1 collects the parameters of the adversary and the coefficients used in its comparison bound. The last two columns distinguish where a parameter's role is introduced from where its value is

chosen and explained. All choices are fixed before the first query. The constants for the finite estimator are given separately in (30)–(33).

Table 1: Parameters and their roles. Section references lead to the explanations in the main text.

Symbol	Value	Role	Introduced	Choice
Adversary parameters				
j	$\log k$	Indexes the degree thresholds; selected from the input size n.	§4.3	§7.1
k	Power of two, $k \geq 64$	Refine a vertex when it has at least this many neighbors.	§3.2	§4
H	2^k	Number of vertices refined in a selected directed path.	§3.2	§4.4
B	$(k+1)^{2H+1}$	Bounds the number of other vertices comparable with one vertex in its in-node DAG during a repair.	§4.1	§4.1
R	2^{2kB}	Sets the width threshold: a coordinate is wide when $W_x \geq 1/R$.	§3.2	§5.3.3
η	$1/(100k)$	Maximum additive error in a probability estimate.	§3.2	§4.5
Comparison-bound coefficients				
s	$(j+10)/k$	Coefficient of the volume and arc terms in the potential.	§4.3	§4.3
a	$s-(s+2)/k$	Coefficient of the anchor level in the refinement inequality.	§4.3	§4.3
C_j	$j+6+4s$	Constant loss term in the refinement inequality.	§4.3	§4.3

References

[1] Mikhail J. Atallah and S. Rao Kosaraju. *An adversary-based lower bound for sorting.* Information Processing Letters 13(2), 55–57, 1981.

[2] Allan Borodin, Leonidas J. Guibas, Nancy A. Lynch, and Andrew C. Yao. *Efficient searching using partial ordering.* Information Processing Letters 12(2), 71–75, 1981. doi:10.1016/0020-0190(81)90005-3.

[3] Dana Richards and Pravin M. Vaidya. *On the distribution of comparisons in sorting algorithms.* BIT 28, 764–774, 1988. Authors' paper.

[4] Gerth Stølting Brodal, Shiva Chaudhuri, and Jaikumar Radhakrishnan. *The randomized complexity of maintaining the minimum.* Nordic Journal of Computing 3(4), 337–351, 1996. Authors' report, BRICS RS-96-40.

[5] Haim Kaplan, Or Zamir, and Uri Zwick. *The amortized cost of finding the minimum.* Proceedings of the Twenty-Sixth Annual ACM–SIAM Symposium on Discrete Algorithms (SODA), 757–768, 2015. Published online 22 December 2014. doi:10.1137/1.9781611973730.51. Authors' paper.

[6] Haim Kaplan, Or Zamir, and Uri Zwick. *A sort of an adversary.* Proceedings of the Thirtieth Annual ACM–SIAM Symposium on Discrete Algorithms (SODA), 1291–1310, 2019. doi:10.1137/1.9781611975482.79.

[7] Richard P. Stanley. *Two poset polytopes.* Discrete & Computational Geometry 1, 9–23, 1986. doi:10.1007/BF02187680. Author's copy.

[8] Graham Brightwell and Peter Winkler. *Counting linear extensions.* Order 8(3), 225–242, 1991. doi:10.1007/BF00383444.

[9] Martin Dyer, Alan Frieze, and Ravi Kannan. *A random polynomial-time algorithm for approximating the volume of convex bodies.* Journal of the ACM 38(1), 1–17, 1991. doi:10.1145/102782.102783. Authors' manuscript.

[10] Jeff Kahn and Jeong Han Kim. *Entropy and sorting.* Journal of Computer and System Sciences 51(3), 390–399, 1995. doi:10.1006/jcss.1995.1077.